

OS Lab8实验报告

练习0：填写已有实验

一、proc.c文件的修改

在proc.c的alloc_proc()函数中，需要初始化进程控制块的各个字段。相比Lab7，Lab8只新加了一个函数的调用，其他没有新增字段，主要是继承之前实验的初始化代码。

```
// LAB4:填写你在lab4中实现的代码
/*
 * Some Useful MACROS, Functions and DEFINES, you can use them in below
 implementation.
 * MACROS or Functions:
 *   local_intr_save():      Disable interrupts
 *   local_intr_restore():   Enable Interrupts
 *   lcr3():                  Modify the value of CR3 register
 *   switch_to():             Context switching between two processes
 */
// 1. 禁用中断
uint32_t flags;
local_intr_save(flags);

// 2. 切换当前进程为要运行的进程
struct proc_struct *prev = current;
current = proc;

// 3. 切换页表，以便使用新进程的地址空间
lcr3(proc->pgdir);

flush_tlb();

// 4. 实现上下文切换
switch_to(&(prev->context), &(proc->context));

// 5. 允许中断
local_intr_restore(flags);
//LAB8 YOUR CODE : (update LAB4 steps)
/*
 * below fields(add in LAB6) in proc_struct need to be initialized
 *   before switch_to();you should flush the tlb
 *   MACROS or Functions:
 *   flush_tlb();          flush the tlb
 */
```

练习1: 完成读文件操作的实现（需要编码）

一、函数功能分析

sfs_io_nolock() 函数的主要功能是：

- 从文件的指定偏移位置读取或写入数据

- 处理块对齐和非对齐的情况
- 支持跨多个磁盘块的读写操作

函数原型：

```
static int sfs_io_nolock(struct sfs_fs *sfs, struct sfs_inode *sin,
                        void *buf, off_t offset, size_t *alenp, bool write)
```

参数说明：

- `sfs`: SFS文件系统结构指针
- `sin`: 内存中的SFS inode结构指针
- `buf`: 读写缓冲区
- `offset`: 文件内的偏移位置
- `alenp`: 输入为需要读写的长度，输出为实际读写的长度
- `write`: 操作类型标志，0表示读，1表示写

二、实现思路

文件I/O操作需要考虑以下三种情况：

1. **第一个不完整的块**：当offset不是块大小的整数倍时，需要处理起始位置的部分块
2. **中间的完整块**：处理完全对齐的块，可以使用块操作函数提高效率
3. **最后一个不完整的块**：当结束位置不是块大小的整数倍时，需要处理末尾的部分块

三、代码实现

在 `sfs_inode.c` 文件的实现了 `sfs_io_nolock()` 函数的核心逻辑：

```
// LAB8 EXERCISE1: 实现文件读写

// (1) 如果 offset 没有与块对齐，先处理第一个不完整的块
blkoff = offset % SFS_BLKSIZE;
if (blkoff != 0) {
    size = (nblks != 0) ? (SFS_BLKSIZE - blkoff) : (endpos - offset);
    if ((ret = sfs_bmap_load_nolock(sfs, sin, blkno, &ino)) != 0) {
        goto out;
    }
    if ((ret = sfs_buf_op(sfs, buf, size, ino, blkoff)) != 0) {
        goto out;
    }
    alen += size;
    buf += size;
    blkno++;
    if (nblks > 0) {
        nblks--;
    }
}

// (2) 处理中间完整对齐的块
while (nblks > 0) {
    if ((ret = sfs_bmap_load_nolock(sfs, sin, blkno, &ino)) != 0) {
```

```

        goto out;
    }
    if ((ret = sfs_block_op(sfs, buf, ino, 1)) != 0) {
        goto out;
    }
    alen += SFS_BLKSIZE;
    buf += SFS_BLKSIZE;
    blkno++;
    nblks--;
}

// (3) 如果结束位置没有与块对齐, 处理最后一个不完整的块
size = endpos % SFS_BLKSIZE;
if (size != 0 && alen < endpos - offset) {
    if ((ret = sfs_bmap_load_nolock(sfs, sin, blkno, &ino)) != 0) {
        goto out;
    }
    if ((ret = sfs_buf_op(sfs, buf, size, ino, 0)) != 0) {
        goto out;
    }
    alen += size;
}

out:
*alenp = alen;
if (offset + alen > sin->din->size) {
    sin->din->size = offset + alen;
    sin->dirty = 1;
}
return ret;

```

四、代码详细解析

1.处理第一个不完整的块

```

blkoff = offset % SFS_BLKSIZE;
if (blkoff != 0) {
    size = (nblks != 0) ? (SFS_BLKSIZE - blkoff) : (endpos - offset);
    if ((ret = sfs_bmap_load_nolock(sfs, sin, blkno, &ino)) != 0) {
        goto out;
    }
    if ((ret = sfs_buf_op(sfs, buf, size, ino, blkoff)) != 0) {
        goto out;
    }
    alen += size;
    buf += size;
    blkno++;
    if (nblks > 0) {
        nblks--;
    }
}

```

实现逻辑:

1. 计算块内偏移 `blkoff = offset % SFS_BLKSIZE`
2. 如果 `blkoff != 0`，说明起始位置未对齐，需要处理部分块
3. 计算需要读写的大小：
 1. 如果还有完整块要处理 (`nblks != 0`)，则读写到块末尾：`SFS_BLKSIZE - blkoff`
 2. 如果这是唯一的块 (`nblks == 0`)，则读写剩余所有数据：`endpos - offset`
4. 调用 `sfs_bmap_load_nolock` 获取物理块号 `ino`
5. 调用 `sfs_buf_op` (读操作为 `sfs_rbuf`，写操作为 `sfs_wbuf`) 进行实际的I/O操作
6. 更新相关变量：累计已处理长度、移动缓冲区指针、块号加1、剩余块数减1

2.处理中间的完整块

```
while (nblks > 0) {
    if ((ret = sfs_bmap_load_nolock(sfs, sin, blkno, &ino)) != 0) {
        goto out;
    }
    if ((ret = sfs_block_op(sfs, buf, ino, 1)) != 0) {
        goto out;
    }
    alen += SFS_BLKSIZE;
    buf += SFS_BLKSIZE;
    blkno++;
    nblks--;
}
```

实现逻辑：

1. 循环处理所有完整对齐的块
2. 对每个块：
 1. 获取物理块号 `ino`
 2. 调用 `sfs_block_op` (读操作为 `sfs_rblock`，写操作为 `sfs_wblock`) 进行整块I/O
 3. 更新累计长度、缓冲区指针、块号
3. 使用块操作函数 (而不是buf操作函数) 可以提高效率，因为不需要处理块内偏移

3.处理最后一个不完整的块

```
size = endpos % SFS_BLKSIZE;
if (size != 0 && alen < endpos - offset) {
    if ((ret = sfs_bmap_load_nolock(sfs, sin, blkno, &ino)) != 0) {
        goto out;
    }
    if ((ret = sfs_buf_op(sfs, buf, size, ino, 0)) != 0) {
        goto out;
    }
    alen += size;
}
```

实现逻辑：

1. 计算结束位置在块内的偏移：`size = endpos % SFS_BLKSIZE`

2. 如果 `size != 0` 且还有数据未处理，说明需要处理最后一个部分块
3. 获取物理块号并调用 `sfs_buf_op` 从块开头读写 `size` 字节
4. 更新累计长度

4.更新文件大小

```
out:
    *alenp = alen;
    if (offset + alen > sin->din->size) {
        sin->din->size = offset + alen;
        sin->dirty = 1;
    }
    return ret;
```

实现逻辑：

1. 将实际处理的长度写回 `*alenp`
2. 如果写操作导致文件变大，更新文件大小并标记inode为脏
3. 返回操作结果

练习2: 完成基于文件系统的执行程序机制的实现（需要编码）

一、整体架构对比

之前的执行流程（内存方式）

```
kernel_execve → do_execve(binary数据) → load_icode(binary) → 直接从内存读取
```

现在的执行流程（文件系统方式）

```
kernel_execve → do_execve(路径) → 打开文件 → load_icode(fd) → load_icode_read → 从文件读取
```

二、关键函数实现

2.1 load_icode 修改

原 `load_icode(binary, size)` 直接从内存缓冲区（`binary` 指针）读取预加载的 ELF 二进制数据，无法支持从文件系统加载可执行文件；新函数 `load_icode(fd, argc, kargv)` 接收文件描述符（`fd`）、命令行参数个数（`argc`）和内核态参数数组（`kargv`），实现从文件系统读取 ELF 文件、加载程序段，并向用户进程传递命令行参数

2.1.1详细实现步骤

（1）内存管理结构创建

为新进程创建独立的内存管理结构（`mm_struct`）和页目录，是进程地址空间隔离的基础

```
// (1) 创建新的内存管理结构
struct mm_struct *mm;
if ((mm = mm_create()) == NULL) {
    goto bad_mm;
}

// (2) 创建页目录
if (setup_pgdir(mm) != 0) {
    goto bad_pgdir_cleanup_mm;
}
```

(2) ELF文件读取与解析

之前的实现方式是直接通过指针偏移访问 ELF 头和程序段头，无法处理 文件存储在磁盘或文件系统 的场景：

```
struct elfhdr *elf = (struct elfhdr *)binary; // 直接指针访问
struct proghdr *ph = (struct proghdr *) (binary + elf->e_phoff);
```

现在的实现方式：

```
// 读取ELF头部
struct elfhdr __elf, *elf = &__elf;
if (load_icode_read(fd, elf, sizeof(struct elfhdr), 0) != 0) {
    goto bad_elf_cleanup_pgdir;
}

// 验证ELF魔数
if (elf->e_magic != ELF_MAGIC) {
    goto bad_elf_cleanup_pgdir;
}

// 逐个读取程序段头部
struct proghdr __ph, *ph = &__ph;
for (int i = 0; i < elf->e_phnum; i++) {
    off_t phoff = elf->e_phoff + sizeof(struct proghdr) * i;
    if ((ret = load_icode_read(fd, ph, sizeof(struct proghdr), phoff)) != 0) {
        goto bad_cleanup_mmap;
    }
    // 处理每个段...
}
```

关键代码解释：

1. 栈上分配临时变量：

1. `struct elfhdr __elf`：在栈上分配 ELF 头结构体，而非直接指向内存缓冲区，避免依赖“二进制数据已在内存”的前提；`elf` 指针指向该栈变量，方便后续操作。
2. 同理，`struct proghdr __ph` 为程序段头的栈上临时存储

2. 按需读取文件数据：

第一步读取 ELF 头：偏移为 0（文件起始位置），长度为 `sizeof(struct elfhdr)`，确保获取 ELF 文件的基本信息。

3. 遍历读取程序段头：

1. `phoff = elf->e_phoff + sizeof(struct proghdr) * i`: 计算第 `i` 个程序段头在文件中的偏移, `e_phoff` 是程序段头表的起始偏移, 每个段头占 `sizeof(struct proghdr)` 字节。
2. 逐个读取每个程序段头, 确保能获取每个段的加载地址、文件偏移、长度、权限等关键信息, 为后续加载程序段做准备。

(3) 程序段加载

将 ELF 文件中的程序段 (代码段、数据段等) 按页读取并加载到进程的虚拟地址空间, 是可执行文件运行的核心步骤。最初的版本直接从内存缓冲区一次性复制整个程序段到目标页面, 无法处理“文件分块读取”和按页分配内存的场景

```
off_t offset = ph->p_offset;
while (start < end) {
    // 分配物理页
    if ((page = pgdir_alloc_page(mm->pgdir, la, perm)) == NULL) {
        goto bad_cleanup_mmap;
    }

    // 计算本次读取的位置和大小
    off = start - la;
    size = PGSIZE - off;
    la += PGSIZE;
    if (end < la) {
        size -= la - end;
    }

    // 从文件读取到页面
    if ((ret = load_icode_read(fd, page2kva(page) + off, size, offset)) != 0) {
        goto bad_cleanup_mmap;
    }

    start += size;
    offset += size; // 更新文件偏移
}
```

关键代码解释:

1. 按页加载的核心逻辑:

1. 程序段的虚拟地址范围是 `[start, end)`, 循环按页拆分该范围, 逐页分配物理内存、读取文件数据。
2. `pgdir_alloc_page(mm->pgdir, la, perm)`: 为虚拟地址 `la` 分配物理页, 并在进程的页目录中建立虚拟地址到物理地址的映射, 同时设置页面权限。

2. 页内偏移计算:

1. `off = start - la`: 若 `start` 不是页对齐地址, `la` 是页起始地址, 则 `off` 为页内偏移, 确保数据写入页面的正确位置。
2. `size = PGSIZE - off`: 本次读取的最大长度为当前页剩余空间, 避免跨页写入; 若为最后一页, 则调整 `size` 为实际需要的长度。

3. 文件偏移跟踪:

`offset` 变量始终记录当前要读取的文件位置, 与程序段的文件偏移同步, 确保从文件的正确位置读取数据到对应虚拟地址。

4. 错误处理:

每次分配物理页、读取文件都检查返回值，失败则跳转到 `bad_cleanup_mmap` 释放已分配的内存映射，避免内存泄漏。

(4) 关闭文件描述符

```
sysfile_close(fd); // 读取完成后关闭文件
```

`sysfile_close(fd)`: 封装的系统调用级文件关闭函数，会递减文件的引用计数，若引用计数为 0 则真正关闭文件。

(5) 用户栈设置

在用户栈上构造 `argc` 和 `argv` 的布局，使用户进程的 `main(int argc, char *argv[])` 能正确获取命令行参数;

1) 用户栈从高地址向低地址增长，最终布局如下:

```
高地址 USTACKTOP
|
+-----+
| 字符串区域      | ← argv[0]的内容 "ls\0"
| argv[0] 字符串  | ← argv[1]的内容 "-l\0"
| argv[1] 字符串  | ← argv[2]的内容 NULL
| ...            |
+-----+ ← stacktop (16字节对齐)
| padding         | ← 对齐填充
+-----+
| argv[0] 指针     | ← 指向字符串区域
| argv[1] 指针     | ←
| argv[2] = NULL   |
+-----+ ← uargv_base (16字节对齐)
| padding         |
+-----+ ← tf->gpr.sp (栈指针)
| 可用栈空间      |
低地址
```

• 实现代码

```
// 计算所有字符串的总长度
uint32_t argv_size = 0;
int i;
for (i = 0; i < argc; i++) {
    argv_size += strlen(kargv[i], EXEC_MAX_ARG_LEN + 1) + 1;
}

// 计算字符串区起始地址 (向下对齐到16字节)
uintptr_t stacktop = USTACKTOP - argv_size;
stacktop &= ~0xF; // 16字节对齐

// argv 指针数组的位置
uintptr_t uargv_base = stacktop - argc * sizeof(char *);
uargv_base &= ~0xF; // 16字节对齐
```



```
// 检查是否超出栈空间
if (uargv_base < USTACKTOP - USTACKSIZE) {
    ret = -E_NO_MEM;
    goto bad_cleanup_mmap;
}
```

代码解释：

- `EXEC_MAX_ARG_LEN`：最大参数长度限制，防止单个参数过长导致栈溢出，是内核安全编程的常用手段。
- 16 字节对齐：RISC-V 的函数调用约定要求栈指针必须是 16 的倍数，否则会导致函数调用异常；`&~0xF` 是将地址的最后 4 位清零，实现 16 字节对齐。
- 栈空间检查：`uargv_base` 是 `argv` 数组的基址，若其低于栈底，说明参数总长度超出用户栈大小，返回内存不足错误。

2) **数据复制**：由于此时页表已经切换到新进程的地址空间，不能直接使用虚拟地址复制，必须通过物理地址访问：

```
uintptr_t str_addr = stacktop;
for (i = 0; i < argc; i++) {
    size_t len = strlen(kargv[i], EXEC_MAX_ARG_LEN + 1) + 1;

    // 方法：逐字节复制（处理跨页情况）
    for (size_t j = 0; j < len; j++) {
        // 获取目标虚拟地址对应的页表项
        pte_t *ptep = get_pte(mm->pgdir, str_addr + j, 0);
        if (ptep == NULL || !(*ptep & PTE_V)) {
            ret = -E_INVALID;
            goto bad_cleanup_mmap;
        }

        // 通过物理地址复制
        struct Page *p = pte2page(*ptep);
        char *dst = (char *)page2kva(p) + ((str_addr + j) & (PGSIZE - 1));
        *dst = kargv[i][j];
    }

    // 设置 argv[i] 指针
    uintptr_t argv_ptr_addr = uargv_base + i * sizeof(char *);
    pte_t *ptep = get_pte(mm->pgdir, argv_ptr_addr, 0);
    if (ptep == NULL || !(*ptep & PTE_V)) {
        ret = -E_INVALID;
        goto bad_cleanup_mmap;
    }

    struct Page *p = pte2page(*ptep);
    char **dst = (char **)((char *)page2kva(p) + (argv_ptr_addr & (PGSIZE - 1)));
    *dst = (char *)str_addr; // 存储用户空间虚拟地址

    str_addr += len;
}
```

1. 逐字节复制的原因：

1. 字符串可能跨越多个物理页，逐字节查页表能确保每个字节写入正确的物理页。
2. 每个字节的虚拟地址可能对应不同的物理页，无法一次性通过指针复制。

2. 虚实地址转换流程：

用户虚拟地址 (`str_addr + j`) → `get_pte()` → 页表项 (`pte`) → `pte2page()` → 物理页 (`Page`) → `page2kva()` → 内核虚拟地址 → 页内偏移 → 最终写入地址

3. `argv` 指针的存储：

`*dst = (char *)str_addr`：存入 `argv` 数组的是用户空间的虚拟地址，而非内核虚拟地址；用户进程运行时，通过该虚拟地址能正确访问字符串区域，若存入内核地址则用户进程无法访问。

(6) 设置 `Trapframe`

设置进程的初始执行上下文，使进程启动后能正确执行 ELF 入口函数，并传递 `argc/argv`。

```
tf->gpr.sp = uargv_base;           // 栈指针指向argv数组
tf->epc = elf->e_entry;

// 设置 argc 和 argv (RISC-V调用约定)
tf->gpr.a0 = argc;                  // 参数1: argc
tf->gpr.a1 = uargv_base;            // 参数2: argv指针

tf->status = (sstatus & ~SSTATUS_SPP) | SSTATUS_SPIE;
```

关键变化：

- 栈指针不再指向 `USTACKTOP`，而是指向 `argv` 数组基址
- 使用 `a0` 和 `a1` 寄存器传递 `argc` 和 `argv`（符合RISC-V调用约定）
- 用户程序的 `main(int argc, char *argv[])` 函数将直接从这些寄存器获取参数

2.2 `do_execve` 修改

`do_execve` 是 `execve` 系统调用的内核处理函数，用于替换当前进程的地址空间、代码和数据为新的可执行文件，保留进程 `PID` 但替换所有执行上下文。在这里我们完善了“参数复制”和“文件操作流程”。

(1) 参数处理

```
// 1. 复制命令行参数到内核空间
char *kargv[EXEC_MAX_ARG_NUM];
const char *path;

lock_mm(mm);
if (name == NULL) {
    snprintf(local_name, sizeof(local_name), "<null> %d", current->pid);
} else {
    if (!copy_string(mm, local_name, name, sizeof(local_name))) {
        unlock_mm(mm);
        return ret;
    }
}

// 复制所有参数
if ((ret = copy_kargv(mm, argc, kargv, argv)) != 0) {
```

```

    unlock_mm(mm);
    return ret;
}
path = argv[0];
unlock_mm(mm);

```

代码解释

- **为什么要复制参数?** 用户态的 `argv` 指针指向的是当前进程的用户虚拟地址，当 `execve` 替换进程地址空间后，原用户虚拟地址会失效；因此必须在销毁旧地址空间前，将参数复制到内核空间（`kargv`），确保后续加载程序时能访问参数
- `lock_mm(mm)` / `unlock_mm(mm)`：内存管理结构的互斥锁，防止多线程并发修改 `mm_struct`，导致数据竞争。
- `copy_string()` / `copy_kargv()`：封装的用户态→内核态字符串复制函数，会检查用户地址的合法性，是内核安全的关键。

(2) 文件系统操作

```

// 1. 关闭当前进程的所有文件
files_closeall(current->filesp);

// 2. 打开可执行文件
int fd;
if ((ret = fd = sysfile_open(path, O_RDONLY)) < 0) {
    goto execve_exit;
}

// 3. 销毁旧地址空间
if (mm != NULL) {
    lsatp/boot_pgdir_pa);
    if (mm_count_dec(mm) == 0) {
        exit_mmap(mm);
        put_pgdir(mm);
        mm_destroy(mm);
    }
    current->mm = NULL;
}

// 4. 加载新程序
if ((ret = load_icode(fd, argc, kargv)) != 0) {
    goto execve_exit;
}

// 5. 清理参数
put_kargv(argc, kargv);

```

执行流程：

1. 关闭所有文件描述符：

`files_closeall(current->filesp)`：关闭当前进程打开的所有文件，避免新程序继承旧的文件描述符；`current` 是指向当前进程 `proc_struct` 的全局指针。

2. 打开新可执行文件：

`sysfile_open(path, O_RDONLY)`：以只读模式打开可执行文件，返回文件描述符；若打开失败，跳转到 `execve_exit` 退出。

3. 销毁旧地址空间：

1. `lsatp(boot_pgdir_pa)`：RISC-V 的 `satp` 寄存器存储页目录的物理地址，切换到内核的启动页目录（`boot_pgdir_pa`），避免后续操作访问已销毁的用户页表导致页错误。
2. 引用计数管理：`mm_count_dec(mm)` 递减 `mm_struct` 的引用计数，若计数为 0，则释放所有相关资源。
4. **加载新程序**：调用重构后的 `load_icode`，从文件描述符 `fd` 加载程序，并传递命令行参数。
5. **清理参数**：`put_kargv()` 释放 `copy_kargv()` 复制到内核的参数字符串，避免内存泄漏。

2.3 do_fork 修改

`do_fork` 是 `fork` 系统调用的内核处理函数，作用是创建一个新进程（子进程），与父进程共享 / 复制资源。在这里我们新增了“文件描述符表的复制 / 共享”逻辑，使子进程能正确继承父进程的文件描述符

(1) 文件结构复制

```
int do_fork(uint32_t clone_flags, uintptr_t stack, struct trapframe *tf)
{
    // ... 原有代码 ...

    // 步骤3：复制或共享内存空间
    if (copy_mm(clone_flags, proc) != 0) {
        goto bad_fork_cleanup_kstack;
    }

    // 复制文件结构
    if (copy_files(clone_flags, proc) != 0) {
        goto bad_fork_cleanup_mm;
    }

    // ... 原有代码 ...

bad_fork_cleanup_mm:
    // 清理内存管理结构
    if (proc->mm != NULL) {
        exit_mmap(proc->mm);
        put_pgdir(proc->mm);
        mm_destroy(proc->mm);
    }
    goto bad_fork_cleanup_kstack;
}
```

代码解释：

- `clone_flags`：创建进程的标志位，如 `CLONE_FS` 表示子进程与父进程共享文件系统相关资源（文件描述符表、当前工作目录等）。
- 错误处理：若复制文件描述符表失败，跳转到 `bad_fork_cleanup_mm` 清理已复制的内存管理结构，保证资源不泄漏。

(2) `copy_files` 实现

```

static int copy_files(uint32_t clone_flags, struct proc_struct *proc)
{
    struct files_struct *filesp, *old_filesp = current->filesp;
    assert(old_filesp != NULL);

    // 如果设置了CLONE_FS标志，共享文件结构
    if (clone_flags & CLONE_FS) {
        filesp = old_filesp;
        goto good_files_struct;
    }

    // 否则创建新的文件结构
    int ret = -E_NO_MEM;
    if ((filesp = files_create()) == NULL) {
        goto bad_files_struct;
    }

    // 复制文件描述符
    if ((ret = dup_files(filesp, old_filesp)) != 0) {
        goto bad_dup_cleanup_fs;
    }

good_files_struct:
    files_count_inc(filesp);
    proc->filesp = filesp;
    return 0;

bad_dup_cleanup_fs:
    files_destroy(filesp);
bad_files_struct:
    return ret;
}

```

设计思想：

- 类似于 `copy_mm`，支持共享或复制两种模式
- 线程（CLONE_FS）共享文件描述符表
- 进程创建独立的文件描述符表

关键函数：

- `files_create()`：分配并初始化空的 `files_struct`。
- `dup_files(filesp, old_filesp)`：将父进程的文件描述符表复制到子进程的文件描述符表，每个文件描述符的引用计数递增。
- `files_count_inc(filesp)`：递增文件结构的引用计数，实现资源的共享计数。

2.4 `alloc_proc` 修改

`alloc_proc` 用于分配并初始化新的进程结构，是进程创建的第一步。在这里我们新增了“文件结构指针初始化”，避免野指针问题。

```
static struct proc_struct *alloc_proc(void)
{
    struct proc_struct *proc = kmalloc(sizeof(struct proc_struct));
    if (proc != NULL) {
        // ... LAB6的初始化代码 ...

        // LAB8新增
        proc->filesp = NULL; // 初始化文件结构指针
    }
    return proc;
}
```

代码解释：

- `proc->filesp`：进程结构体中指向文件描述符表的指针；初始化为 `NULL` 能避免后续操作访问野指针，导致系统崩溃。
- 内核编程规范：所有指针类型的成员在初始化时必须显式赋值，避免未初始化的指针指向随机内存地址，引发不可预测的错误。

四、关键技术点总结

4.1 从内存到文件系统的转变

方面	内存	文件系统
数据源	内存中的二进制数据	文件系统上的文件
访问方式	直接指针访问	文件I/O操作
读取模式	随机访问	seek + read
错误处理	简单	需要处理I/O错误

4.2 命令行参数传递机制

内核空间 `kargv[]` → 复制到 → 用户栈 → 设置寄存器 `a0/a1` → 用户程序 `main()`

关键步骤：

1. 从用户空间复制参数到内核（`copy_kargv`）
2. 在新进程的用户栈构造 `argc/argv`
3. 通过 `trapframe` 的 `a0/a1` 寄存器传递参数
4. 用户程序从寄存器获取参数

4.3 地址空间切换的时序

1. lock_mm, 复制参数到内核
2. unlock_mm
3. 打开文件
4. 销毁旧地址空间, 切换到内核页表
5. 创建新地址空间
6. 通过物理地址复制参数到用户栈
7. 切换到新地址空间

为什么这个顺序很重要?

- 参数复制必须在旧地址空间销毁前完成
- 新地址空间的用户栈写入必须通过物理地址

4.4 错误处理的完整性

使用 `goto` 标签实现清理:

```
bad_cleanup_mmap:      // 清理内存映射
    exit_mmap(mm);
bad_elf_cleanup_pgdir: // 清理页目录
    put_pgdir(mm);
bad_pgdir_cleanup_mm:  // 清理mm结构
    mm_destroy(mm);
bad_mm:
    return -E_NO_MEM;
```

每个错误点都能正确回滚已分配的资源。

五、实验验证

输入命令 `make qemu` 后, 成功进入了sh用户程序的执行界面:

```
kernel_execve: pid = 2, name = "sh"
user sh is running!!!
```

输入 `hello`, 成功执行了hello程序, **表明实验成功!**

```
Initrd: 0xc0214010 - 0xc021bd0f, size: 0x00007d00
Initrd: 0xc021bd10 - 0xc029100f, size: 0x00075300
sfs: mount: 'simple file system' (106/11/117)
vfs: mount disk0.
++ setup timer interrupts
kernel_execve: pid = 2, name = "sh".
user sh is running!!!
Hello world!!.
I am process 3.
hello pass.
$
```

执行 `make grade`, **最终得分100/100。**

```
gmake[1]: 进入目录 "/home/www/riscv/lab8" + cc tools/mksfs.c + cc user/badarg.c + cc user/libs/dir.c + cc user/libs/file.c + cc user/libs/initcode.S + cc user/libs/panic.c + cc user/libs/stdio.c + cc user/libs/syscall.c + cc user/libs/ulib.c + cc user/libs/umain.c + cc libs/hash.c + cc libs/printfmt.c + cc libs/rand.c + cc libs/string.c + cc user/badsegment.c + cc user/divzero.c + cc user/ex.it.c + cc user/faultread.c + cc user/faultreadkernel.c + cc user/forktest.c + cc user/forktree.c + cc user/hello.c + cc user/matrix.c + cc user/pgdir.c + cc user/priority.c + cc user/sh.c + cc user/sleep.c + cc user/sleepkill.c + cc user/softint.c + cc user/spin.c + cc user/testbss.c + cc user/waitkill.c + cc user/yield.c create bin/sfs.img (disk0) successfully. + cc kern/init/entry.S + cc kern/init/init.c + cc kern/libs/readline.c + cc kern/libs/stdio.c + cc kern/libs/string.c + cc kern/debug/kdebug.c + cc kern/debug/kmonitor.c + cc kern/debug/panic.c + cc kern/driver/clock.c + cc kern/driver/console.c + cc kern/driver/dtb.c + cc kern/driver/ide.c + cc kern/driver/intr.c + cc kern/driver/picirq.c + cc kern/driver/ramdisk.c + cc kern/trap/trap.c + cc kern/trap/trapentry.S + cc kern/mm/default_pmm.c + cc kern/mm/kmalloc.c + cc kern/mm/pmm.c + cc kern/mm/vmm.c + cc kern/sync/check_sync.c + cc kern/sync/monitor.c + cc kern/sync/sem.c + cc kern/sync/wait.c + cc kern/sfs/file.c + cc kern/sfs/fs.c + cc kern/sfs/iobuf.c + cc kern/sfs/sysfile.c + cc kern/process/entry.S + cc kern/process/proc.c + cc kern/process/switch.S + cc kern/schedule/default_sched.c + cc kern/schedule/default_sched_stride.c + cc kern/schedule/sched.c + cc kern/syscall/syscall.c + cc kern/fs/swap/swapsfs.c + cc kern/fs/vfs/inode.c + cc kern/fs/vfs/vfs.c + cc kern/fs/vfs/vfsdev.c + cc kern/fs/vfs/vfsfile.c + cc kern/fs/vfs/vfslookup.c + cc kern/fs/vfs/vfspath.c + cc kern/fs/devs/dev.c + cc kern/fs/devs/dev_disk0.c + cc kern/fs/devs/dev_stdin.c + cc kern/fs/devs/dev_stdout.c + cc kern/fs/sfs/bitmap.c + cc kern/fs/sfs/sfs.c + cc kern/fs/sfs/sfs_fs.c + cc kern/fs/sfs/sfs_inode.c + cc kern/fs/sfs/sfs_io.c + cc kern/fs/sfs/sfs_lock.c + ld bin/kernel riscv64-unknown-elf-objcopy bin/kernel --strip-all -O binary bin/ucore.img gmake[1]: 离开目录 "/home/www/riscv/lab8"
-sh execve: OK
-user sh : OK

Total score: 100/100

www@www-Virtual-Platform:~/riscv/lab8$
```

六、实验中遇到的问题及解决方案

6.1 问题描述: Store/AMO Page Fault 错误

问题现象

实验过程中，系统成功进入了 shell 界面后，但此时按下任意键都会持续输出 `store/AMO page fault` 错误，导致系统无法正常使用。

```
Store/AMO page fault
Store/AMO page fault
Store/AMO page fault
... (无限循环)
```

[illegible]

问题定位

通过代码审查，发现问题出在 `load_icode` 函数中处理 ELF 程序段时的逻辑：

```
// 原始代码（有问题的版本）
for (int i = 0; i < elf->e_phnum; i++) {
    // ... 读取程序段头 ...
}
```



```

    if (ph->p_type != ELF_PT_LOAD) {
        continue;
    }
    if (ph->p_filesz > ph->p_memsz) {
        goto bad_cleanup_mmap;
    }
    if (ph->p_filesz == 0) {
        continue; // ❌ 这里有问题!
    }

    // ... 后续处理代码 ...
}

```

6.2 问题根因分析

BSS段的特殊性

1. 什么是BSS段?

BSS段用于存储未初始化的全局变量和静态变量

2. BSS段在ELF文件中的表示

```

// BSS段的特点:
p_filesz = 0      // 文件中不占空间（无需存储0值）
p_memsz > 0      // 内存中需要空间
p_offset = 某个值 // 文件偏移可能无意义

```

3. 为什么 `p_filesz == 0`?

为了节省磁盘空间，未初始化的数据不需要存储在可执行文件中，并且加载时只需要在内存中分配空间并清零即可。

错误的执行流程

原始代码逻辑：

1. 读取程序段头，发现 BSS 段 (`p_filesz == 0`)
- ↓
2. 执行 `continue`，跳过此段
- ↓
3. ❌ 未调用 `mm_map()` → BSS段没有虚拟内存映射
- ↓
4. ❌ 未调用 `pgdir_alloc_page()` → BSS段没有物理页面
- ↓
5. 用户程序开始执行
- ↓
6. 程序访问全局变量（位于BSS段）
- ↓
7. ⚠️ CPU查页表，发现虚拟地址没有映射
- ↓
8. ✨ 触发 `Store/AMO Page Fault`（存储访问页错误）

触发过程：

编译器将 `global_count` 放在 BSS 段，ELF 文件中，BSS 段的 `p_filesz = 0`，在 `load_icode` 中因为 `p_filesz == 0` 而跳过了 BSS 段，没有为 BSS 段建立页表映射，程序执行 `global_count++` 时，CPU 尝试访问 BSS 段地址，MMU 查找页表，发现该地址没有映射，于是触发 Store/AMO page fault。

6.3 解决方案

方法：注释掉或删除错误的 `continue` 语句

```
if (ph->p_filesz == 0) {  
    //continue; // 注释掉  
}  
// 或者直接删除整个if块
```

修改后的完整逻辑：

```
// 即使 p_filesz == 0，仍然需要：  
// 1. 调用 mm_map() 建立虚拟内存映射  
// 2. 在后续的 BSS 处理代码中分配物理页面并清零  
// 3. 确保程序能够正常访问 BSS 段的变量
```

6.4 验证与测试

修改后的行为：

1. 正确处理BSS段

```
加载流程：  
检测到 BSS 段 (p_filesz = 0, p_memsz = 0x1000)  
↓  
调用 mm_map(0x10000, 0x1000, ...) 建立VMA  
↓  
跳过文件读取（因为 p_filesz = 0）  
↓  
进入 BSS 清零代码  
↓  
调用 pgdir_alloc_page() 分配页面  
↓  
调用 memset(..., 0, ...) 清零  
↓  
程序访问 global_count 时能够正常读写
```

2. 系统正常运行

1. Shell 可以正常接收键盘输入
2. 用户程序能够访问全局变量
3. 不再出现 page fault 错误

扩展练习 Challenge1：基于"UNIX的PIPE机制"的设计方案

一、概述

管道（Pipe）是 UNIX 系统中最基本的进程间通信机制之一。管道本质上是一个内核缓冲区，允许一个进程向其中写入数据，另一个进程从中读取数据，实现单向的数据流传输。

在 Linux 系统中，管道通过 `pipe()` 系统调用创建，返回两个文件描述符，分别对应读端和写端。管道的实现依赖于内核中的环形缓冲区，配合信号量机制实现进程间的同步与互斥。本设计方案参考了 Linux 内核的管道实现，针对 uCore 的特点进行了适当简化。

二、设计目标

- 实现匿名管道，支持父子进程间通信
- 提供标准的读写接口，与文件操作接口统一
- 正确处理同步互斥问题，保证数据一致性
- 处理管道满/空时的阻塞与唤醒机制

三、数据结构设计

3.1 管道核心结构

管道的核心数据结构 `pipe_inode` 是整个管道机制的基础，该结构体包含了管道运行所需的所有状态信息。在设计这个结构体时，我们需要考虑三个核心问题：数据如何存储、如何追踪管道的使用状态、以及如何实现进程间的同步。

首先是**数据存储部分**。我们使用一个固定大小的环形缓冲区 `buffer`，大小设为 4096 字节。`read_pos` 和 `write_pos` 分别记录下一次读取和写入的位置，它们在缓冲区中循环移动，形成环形队列的效果。`data_size` 记录当前缓冲区中有效数据的字节数，这个字段使得判断缓冲区空满状态变得简单高效。

其次是**引用计数部分**。`readers` 和 `writers` 分别记录当前打开的读端和写端数量。这两个计数器对于管道的正确语义非常重要：

- 当 `writers` 降为 0 时，所有读操作应该返回 EOF（文件结束）；
- 当 `readers` 降为 0 时，所有写操作应该收到 SIGPIPE 信号。
- `ref_count` 是总引用计数，当它降为 0 时，整个管道结构可以被释放。

最后是**同步机制部分**。我们使用三个信号量来协调读写进程：`mutex` 是互斥信号量，确保同一时刻只有一个进程能够操作缓冲区；`not_full` 是条件信号量，写进程在缓冲区满时等待这个信号量；`not_empty` 也是条件信号量，读进程在缓冲区空时等待这个信号量。此外，`read_wait` 和 `write_wait` 两个等待队列用于更精细的进程唤醒控制。

```
#define PIPE_BUF_SIZE    4096    // 管道缓冲区大小（与Linux一致）

/* 管道结构体 */
struct pipe_inode {
    char buffer[PIPE_BUF_SIZE];    // 环形缓冲区
    uint32_t read_pos;            // 读位置指针
    uint32_t write_pos;           // 写位置指针
    uint32_t data_size;           // 当前缓冲区中的数据量
```

```

uint32_t readers;           // 读端引用计数
uint32_t writers;          // 写端引用计数

semaphore_t mutex;         // 互斥信号量，保护缓冲区操作
semaphore_t not_full;      // 条件信号量：缓冲区非满
semaphore_t not_empty;     // 条件信号量：缓冲区非空

wait_queue_t read_wait;    // 读等待队列
wait_queue_t write_wait;   // 写等待队列

int ref_count;             // 总引用计数
};

```

3.2 管道文件结构

管道虽然不是传统意义上的磁盘文件，但它同样**通过文件描述符来访问**，所以我们使用了相同的 `read()`、`write()`、`close()` **系统调用**。为了实现这种统一的抽象，我们需要扩展 uCore 现有的 `file` 结构体，使其能够表示管道类型的文件。

扩展后的 `file` 结构体引入了 `fd_type` 枚举字段，用于区分不同类型的文件：

- `FD_REGULAR` 表示普通磁盘文件
- `FD_PIPE` 表示管道
- `FD_DEVICE` 表示设备文件

VFS 层在处理文件操作时，会根据这个字段选择不同的处理函数。`fd_flags` 字段对于管道特别重要，它使用 `FD_PIPE_READ` 和 `FD_PIPE_WRITE` 标志来区分当前文件描述符是管道的读端还是写端，因为同一个管道的读写端在行为上有本质区别。

`fd_readable` 和 `fd_writable` 字段明确了文件描述符的权限。对于管道读端，`fd_readable` 为真而 `fd_writable` 为假；对于管道写端则相反。

```

/* 管道文件类型标识 */
#define FD_PIPE_READ    0x01    // 管道读端
#define FD_PIPE_WRITE   0x02    // 管道写端

/* 扩展文件结构以支持管道 */
struct file {
    enum {
        FD_NONE,
        FD_REGULAR,           // 普通文件
        FD_PIPE,              // 管道文件
        FD_DEVICE             // 设备文件
    } fd_type;

    int fd_flags;              // 文件标志（包含管道读/写端标识）
    int fd_readable;           // 是否可读
    int fd_writable;           // 是否可写

    union {
        struct inode *fd_inode; // 普通文件的inode
        struct pipe_inode *fd_pipe; // 管道结构指针
    };
};

```

```

off_t fd_offset;           // 文件偏移（管道不使用）
int ref_count;             // 引用计数
};

```

3.3 管道文件操作结构

为了让 VFS 层能够统一处理不同类型的文件，我们采用了面向对象的设计模式——通过函数指针表来实现多态。pipe_operations 结构体定义了管道支持的所有操作，每个字段都是一个函数指针，指向具体的实现函数。当 VFS 层需要对管道执行某个操作时，它会通过这个函数指针表调用相应的函数，而不需要知道管道内部的实现细节。

read 函数指针指向管道的**读取实现**，它从环形缓冲区中取出数据并复制到用户空间。write 函数指针指向管道的**写入实现**，它将用户空间的数据复制到环形缓冲区中。这两个函数的签名与普通文件的读写操作相似，但内部实现完全不同——**管道操作的是内存缓冲区，而普通文件操作的是磁盘块**。

release 函数用于关闭管道的一个端点。它需要一个 is_write_end 参数来区分关闭的是读端还是写端，因为两者的处理逻辑有显著差异：关闭写端可能需要唤醒等待数据的读者，关闭读端可能需要唤醒等待空间的写者。poll 函数支持 select/poll/epoll 系统调用，它检查管道当前是否可读或可写，而不实际执行读写操作，这对于实现非阻塞 I/O 多路复用至关重要。

```

/* 管道操作函数表 */
struct pipe_operations {
    ssize_t (*read)(struct pipe_inode *pipe, void *buf, size_t count);
    ssize_t (*write)(struct pipe_inode *pipe, const void *buf, size_t count);
    int (*release)(struct pipe_inode *pipe, int is_write_end);
    unsigned int (*poll)(struct pipe_inode *pipe, int is_write_end);
};

```

四、接口设计

4.1 系统调用接口

sys_pipe 是用户程序创建管道的唯一入口，它是一个系统调用，会陷入内核态执行。

在内核实现中，sys_pipe 需要完成以下步骤：首先调用 pipe_alloc() 分配并初始化一个 pipe_inode 结构；然后在当前进程的文件描述符表中分配两个空闲槽位；接着创建两个 file 结构，分别代表读端和写端，并让它们都指向同一个 pipe_inode；最后将文件描述符返回给用户空间。如果任何一步失败，需要回滚之前的操作，释放已分配的资源。

```

/**
 * sys_pipe - 创建管道
 * @fd: 用于返回两个文件描述符的数组
 *      fd[0] - 管道读端
 *      fd[1] - 管道写端
 *
 * 语义：创建一个匿名管道，返回两个文件描述符。
 *      数据从写端(fd[1])写入，从读端(fd[0])读出。
 * 返回：成功返回0，失败返回-1并设置errno
 */
int sys_pipe(int fd[2]);

```

pipe_read 是管道读取的核心实现。它的语义是：从管道缓冲区读取最多 **count** 字节的数据到用户缓冲区 **buf** 中，返回实际读取的字节数。如果管道当前为空但仍有写端打开，读操作会阻塞，直到有数据可读或所有写端关闭。如果管道为空且所有写端已关闭，函数返回 0，表示文件结束（EOF）。这种语义使得读取管道的程序可以用与读取普通文件相同的方式检测数据流结束。

```
/**
 * pipe_read - 从管道读取数据
 * @pipe: 管道结构指针
 * @buf: 用户缓冲区
 * @count: 请求读取的字节数
 *
 * 语义：从管道缓冲区读取最多count字节到buf中。
 *      若管道为空且有写端打开，则阻塞等待。
 *      若管道为空且所有写端已关闭，返回0（EOF）。
 * 返回：实际读取的字节数，错误返回-1
 */
ssize_t pipe_read(struct pipe_inode *pipe, void *buf, size_t count);
```

pipe_write 是管道写入的核心实现。它将用户缓冲区中的数据写入管道，返回实际写入的字节数。如果管道已满，写操作会阻塞，直到有空间可用。这里有一个重要的边界情况：如果所有读端都已关闭，写操作没有意义，此时内核会向写进程发送 SIGPIPE 信号，函数返回 -EPIPE 错误。

```
/**
 * pipe_write - 向管道写入数据
 * @pipe: 管道结构指针
 * @buf: 用户数据缓冲区
 * @count: 请求写入的字节数
 *
 * 语义：将buf中的数据写入管道缓冲区。
 *      若管道已满，则阻塞等待。
 *      若所有读端已关闭，向写进程发送SIGPIPE信号。
 * 返回：实际写入的字节数，错误返回-1
 */
ssize_t pipe_write(struct pipe_inode *pipe, const void *buf, size_t count);
```

pipe_close 负责关闭管道的一个端点。关闭操作需要正确更新引用计数：如果关闭的是读端，减少 **readers** 计数；如果关闭的是写端，减少 **writers** 计数。当某一端的计数降为 0 时，需要唤醒在另一端等待的进程。当 **ref_count** 降为 0 时，表示管道完全不再被使用，应该释放 **pipe_inode** 占用的内存。

```
/**
 * pipe_close - 关闭管道端点
 * @pipe: 管道结构指针
 * @is_write_end: 是否为写端
 *
 * 语义：关闭管道的读端或写端，减少对应的引用计数。
 *      当读写端引用计数都为0时，释放管道资源。
 * 返回：成功返回0
 */
int pipe_close(struct pipe_inode *pipe, int is_write_end);
```

4.2 内核内部接口

除了用户可见的系统调用，管道机制还需要一组内核内部使用的辅助函数。这些函数不直接暴露给用户程序，但对于管道的正确实现至关重要。

`pipe_alloc` 是管道的"构造函数"，负责分配和初始化 `pipe_inode` 结构。初始化过程包括：将缓冲区清零，将 `read_pos`、`write_pos`、`data_size` 都设为 0，将 `readers` 和 `writers` 都设为 1（因为创建时会同时打开读写两端），初始化三个信号量，以及初始化两个等待队列。这个函数的正确性直接影响管道后续所有操作的行为。

```
/**
 * pipe_alloc - 分配并初始化管道结构
 *
 * 语义：分配pipe_inode结构，初始化缓冲区、信号量、等待队列
 * 返回：成功返回pipe_inode指针，失败返回NULL
 */
struct pipe_inode *pipe_alloc(void);
```

`pipe_free` 是管道的"析构函数"，在管道不再被使用时释放其占用的内存。这个函数相对简单，但调用时机很重要——只有当 `ref_count` 降为 0 时才能调用，否则会导致悬空指针和内存访问错误。在多进程共享管道的场景下，必须等所有进程都关闭了它们的管道端点后，才能释放管道。

```
/**
 * pipe_free - 释放管道资源
 * @pipe: 管道结构指针
 *
 * 语义：释放管道占用的内存资源
 */
void pipe_free(struct pipe_inode *pipe);
```

`pipe_is_empty` 和 `pipe_is_full` 是两个简单的谓词函数，分别判断管道缓冲区是否为空或已满。它们被读写操作频繁调用，用于决定是否需要进行阻塞当前进程。这两个函数只需检查 `data_size` 是否为 0 或等于 `PIPE_BUF_SIZE`，实现非常简单。

```
/**
 * pipe_is_empty - 检查管道是否为空
 * @pipe: 管道结构指针
 *
 * 语义：判断管道缓冲区中是否有数据
 * 返回：空返回1，非空返回0
 */
int pipe_is_empty(struct pipe_inode *pipe);

/**
 * pipe_is_full - 检查管道是否已满
 * @pipe: 管道结构指针
 *
 * 语义：判断管道缓冲区是否已满
 * 返回：满返回1，未满返回0
 */
int pipe_is_full(struct pipe_inode *pipe);
```

五、同步互斥问题处理

5.1 互斥访问

管道缓冲区是一个典型的共享资源，可能被多个进程并发访问。

我们使用互斥信号量 `mutex` 来保护对缓冲区的所有访问。`mutex` 的初始值为 1，表示资源可用。当一个进程想要访问缓冲区时，首先调用 `down(&pipe->mutex)` 尝试获取锁。如果锁已被其他进程持有，当前进程会阻塞在这个操作上，直到锁被释放。访问完成后，进程调用 `up(&pipe->mutex)` 释放锁，这可能会唤醒一个等待锁的进程。

下面的代码片段展示了互斥保护的基本模式。所有对 `read_pos`、`write_pos`、`data_size` 的读写，以及对 `buffer` 的数据复制，都必须在这对 `down/up` 之间进行。这种模式确保了临界区内的操作是原子的，不会被其他进程的操作打断。

```
// 读写操作前后的互斥保护
down(&pipe->mutex);           // 进入临界区
// ... 读写缓冲区操作 ...
up(&pipe->mutex);             // 离开临界区
```

5.2 同步机制

仅有互斥是不够的，我们还需要同步机制来协调读写进程的执行顺序。管道的读写操作有天然的依赖关系：读者需要等待写者提供数据，写者需要等待读者腾出空间。这是经典的生产者-消费者问题，我们使用两个条件信号量 `not_empty` 和 `not_full` 来解决。

`pipe_read` 函数展示了读者的同步逻辑。进入函数后首先获取互斥锁，然后进入一个 `while` 循环检查缓冲区是否为空。使用 `while` 而非 `if` 是为了处理“虚假唤醒”——进程可能在条件不满足时被唤醒，必须重新检查条件。如果缓冲区为空，需要进一步判断：若 `writers` 为 0，说明所有写端都已关闭，不会再有数据到来，返回 0 表示 EOF；否则释放互斥锁，在 `not_empty` 信号量上等待。被唤醒后重新获取互斥锁，再次检查条件。成功读取数据后，调用 `up(&pipe->not_full)` 通知可能在等待空间的写者。

```
/* 读操作的同步处理 */
ssize_t pipe_read(struct pipe_inode *pipe, void *buf, size_t count) {
    down(&pipe->mutex);

    // 管道为空时等待
    while (pipe->data_size == 0) {
        if (pipe->writers == 0) {
            // 没有写端，返回EOF
            up(&pipe->mutex);
            return 0;
        }
        up(&pipe->mutex);
        down(&pipe->not_empty);    // 等待非空信号
        down(&pipe->mutex);
    }

    // 从缓冲区读取数据
    ssize_t bytes_read = do_pipe_read(pipe, buf, count);

    up(&pipe->not_full);           // 唤醒等待写入的进程
    up(&pipe->mutex);
}
```



```

    return bytes_read;
}

```

`pipe_write` 函数的逻辑与读者对称。写者首先检查是否还有读端打开——如果没有，写入的数据无人接收，应该向当前进程发送 `SIGPIPE` 信号并返回错误。然后检查缓冲区是否有足够空间，如果没有则在 `not_full` 信号量上等待。注意在等待期间必须释放互斥锁，否则读者无法读取数据来腾出空间，造成死锁。写入数据后，调用 `up(&pipe->not_empty)` 通知可能在等待数据的读者。

```

/* 写操作的同步处理 */
ssize_t pipe_write(struct pipe_inode *pipe, const void *buf, size_t count) {
    down(&pipe->mutex);

    // 检查是否有读端
    if (pipe->readers == 0) {
        up(&pipe->mutex);
        // 发送SIGPIPE信号给当前进程
        do_signal(current, SIGPIPE);
        return -EPIPE;
    }

    // 管道满时等待
    while (PIPE_BUF_SIZE - pipe->data_size < count) {
        up(&pipe->mutex);
        down(&pipe->not_full);    // 等待非满信号
        down(&pipe->mutex);

        // 再次检查读端
        if (pipe->readers == 0) {
            up(&pipe->mutex);
            return -EPIPE;
        }
    }

    // 向缓冲区写入数据
    ssize_t bytes_written = do_pipe_write(pipe, buf, count);

    up(&pipe->not_empty);    // 唤醒等待读取的进程
    up(&pipe->mutex);

    return bytes_written;
}

```

5.3 引用计数管理

管道端点的关闭处理是整个同步机制中最微妙的部分。当一个进程关闭管道的一端时，不仅要正确更新引用计数，还可能需要唤醒在另一端等待的进程，因为它们等待的条件可能永远不会满足了。

`pipe_close` 函数处理这种情况。首先获取互斥锁，然后根据 `is_write_end` 参数判断关闭的是哪一端。如果关闭的是写端，减少 `writers` 计数；如果降为 0，说明这是最后一个写端，需要唤醒所有等待数据的读者——它们应该检测到 EOF 并正常返回。唤醒操作通过 `wakeup_all(&pipe->read_wait)` 和 `up(&pipe->not_empty)` 两种方式进行，确保所有等待的读者都能被唤醒。

类似地，如果关闭的是读端，减少 `readers` 计数；如果降为 0，需要唤醒所有等待空间的写者——它们应该检测到 EPIPE 条件并返回错误。最后检查 `ref_count`：如果降为 0，说明管道已完全不被使用，可以安全释放。注意释放前要先释放互斥锁，因为 `pipe_free` 会销毁整个结构，包括其中的信号量。

```
/* 关闭管道端点时的处理 */
int pipe_close(struct pipe_inode *pipe, int is_write_end) {
    down(&pipe->mutex);

    if (is_write_end) {
        pipe->writers--;
        if (pipe->writers == 0) {
            // 最后一个写端关闭，唤醒所有等待的读者
            wakeup_all(&pipe->read_wait);
            up(&pipe->not_empty);    // 让读者检测到EOF
        }
    } else {
        pipe->readers--;
        if (pipe->readers == 0) {
            // 最后一个读端关闭，唤醒所有等待的写者
            wakeup_all(&pipe->write_wait);
            up(&pipe->not_full);
        }
    }

    pipe->ref_count--;

    if (pipe->ref_count == 0) {
        up(&pipe->mutex);
        pipe_free(pipe);
        return 0;
    }

    up(&pipe->mutex);
    return 0;
}
```

六、实现要点总结

在完成管道机制的设计后，我们总结以下几个关键的实现要点：

- 1. 环形缓冲区：**使用环形缓冲区实现高效的数据存取，避免数据搬移的开销。读写位置通过取模运算 $(pos + n) \% PIPE_BUF_SIZE$ 在缓冲区中循环移动，形成逻辑上的环形结构。
- 2. 原子性保证：**POSIX 标准要求小于 PIPE_BUF 的写操作具有原子性，即不会与其他写操作交错。这需要在实现中确保小写入操作在持有锁的情况下一一次性完成。
- 3. 非阻塞模式：**除了默认的阻塞模式，还应支持 O_NONBLOCK 标志。在非阻塞模式下，如果操作无法立即完成，应返回 EAGAIN 错误而非阻塞进程。
- 4. 与 VFS 集成：**通过 `file_operations` 结构与虚拟文件系统统一接口，使管道可以像普通文件一样被操作，支持 `read()`、`write()`、`close()`、`poll()` 等标准系统调用。

扩展练习 Challenge2：基于"UNIX的软连接和硬连接机制"的设计方案

一、概述

UNIX 文件系统支持两种链接机制，它们都允许一个文件拥有多个名称，但实现原理和使用场景截然不同。

- 硬链接：**该机制的本质是在目录中创建一个新的目录项，该目录项指向一个已存在文件的 inode。新创建的目录项和原有的目录项完全平等，它们共享同一个 inode，因此共享文件的所有属性：数据内容、权限、所有者、时间戳等。从文件系统的角度看，这两个"文件"实际上是同一个文件的两个名字。删除其中任何一个只是减少 inode 的链接计数，只有当链接计数降为 0 且没有进程打开该文件时，文件数据才会被真正删除。
- 软链接/符号链接：**该机制的本质是创建一个新的特殊文件，该文件的内容是目标文件的路径字符串。当系统访问符号链接时，会读取其中存储的路径，然后重定向到该路径指向的文件。符号链接和目标文件是两个独立的 inode，有各自的权限、所有者等属性。如果删除目标文件，符号链接将变成"悬空链接"（dangling link），访问它会报错。

二、硬链接与软链接的区别

下表详细对比了硬链接和软链接在各个方面的差异，这些差异直接影响到我们的实现策略：

表格 还在加载中，请等待加载完成后再次尝试复制

三、数据结构设计

3.1 扩展 inode 结构

为了支持链接机制，我们需要扩展 uCore 的磁盘 inode 结构 `sfs_disk_inode`。这个结构体存储在磁盘上，描述一个文件的所有元数据。原有结构已经包含了文件大小、类型、数据块索引等基本信息，我们需要添加链接相关的字段。

最关键的新增字段是 `nlinks` **（硬链接计数）。**这个 16 位整数记录了指向该 inode 的目录项数量。当 `nlinks` 降为 0 时，表示没有任何目录项指向该 inode，文件的数据块可以被回收，inode 本身也可以被释放供新文件使用。

`type` 字段用于区分不同类型的文件。我们新增了 `SFS_TYPE_LINK` 类型来标识符号链接。符号链接在很多方面与普通文件相似，但系统在访问时会特殊处理：读取符号链接的内容并将其解释为路径，然后重定向到该路径。**对于符号链接，**`size`** 字段存储的是目标路径字符串的长度，而非文件数据的大小。**

我们还添加了完整的 UNIX 文件属性：`uid` 和 `gid` 记录文件的所有者和所属组；`mode` 存储文件的访问权限以及特殊标志；`atime`、`mtime`、`ctime` 分别记录最后访问时间、最后修改时间和状态改变时间。创建或删除硬链接会改变 inode 的 `nlinks` 字段，这属于状态改变，需要更新 `ctime`。

```
/* 文件类型定义 */
#define SFS_TYPE_FILE      0x01    // 普通文件
#define SFS_TYPE_DIR      0x02    // 目录
#define SFS_TYPE_LINK     0x03    // 符号链接（软链接）

/* 扩展后的磁盘inode结构 */
struct sfs_disk_inode {
```

```

uint32_t size;                // 文件大小（软链接时为路径长度）
uint16_t type;                // 文件类型
uint16_t nlinks;              // 硬链接计数
uint32_t blocks;              // 占用的块数
uint32_t direct[SFS_NDIRECT]; // 直接块索引
uint32_t indirect;            // 间接块索引

/* 以下为新增字段 */
uint32_t uid;                  // 所有者用户ID
uint32_t gid;                  // 所有者组ID
uint32_t mode;                 // 访问权限
uint32_t atime;                // 最后访问时间
uint32_t mtime;                // 最后修改时间
uint32_t ctime;                // 状态改变时间
};

```

3.2 符号链接专用结构

符号链接需要存储目标路径字符串。为了高效处理这不同路径情况，我们采用了两种存储策略。

- **短路径：我们使用"内联存储"：**直接将路径字符串存储在 inode 结构内部，不需要分配额外的数据块。
- **长路径：我们退回到常规的数据块存储：**分配一个或多个数据块来存储路径字符串，inode 中只存储数据块的编号。

`sfs_symlink_data` 结构体使用 **union** 来封装这两种存储方式：`inline_path` 数组用于内联存储，`data_block` 字段用于指向外部数据块。`path_len` 字段记录路径的实际长度，用于判断使用哪种存储方式以及读取时的边界检查。

`sfs_inode_symlink` 是符号链接在内存中的表示。`target_path` 指向一个缓存的目标路径字符串，避免每次访问符号链接都要读取磁盘。`resolved` 标志指示这个缓存是否有效——当符号链接被修改时，需要将 `resolved` 设为假，下次访问时重新从磁盘读取。

```

#define MAX_SYMLINK_LEN      4096    // 符号链接最大长度
#define SYMLINK_INLINE_LEN   60      // 内联存储的最大长度

/* 符号链接数据存储 */
struct sfs_symlink_data {
    uint32_t path_len;                // 目标路径长度

    union {
        char inline_path[SYMLINK_INLINE_LEN]; // 短路径内联存储
        uint32_t data_block;                  // 长路径使用数据块
    };
};

/* 内存中的符号链接inode */
struct sfs_inode_symlink {
    struct sfs_disk_inode din;        // 磁盘inode
    char *target_path;                // 缓存的目标路径
    int resolved;                     // 是否已解析
};

```

3.3 目录项结构

目录项是连接文件名和 inode 的桥梁。每个目录项包含两个关键信息：一个 inode 编号和一个文件名。

硬链接的实现完全基于这个结构：创建硬链接就是在目标目录中添加一个新的目录项，其 inode 编号指向已存在文件的 inode。这个新目录项和原有的目录项在结构上完全相同，唯一的区别是文件名不同。文件系统不需要知道哪个目录项是"原始"的，哪个是"链接"——它们是完全平等的。

值得注意的是，**目录项结构本身不需要为链接机制做任何修改。**软链接虽然概念上不同，但在存储层面也是通过目录项来引用的：创建软链接会分配一个新的 inode，然后在目录中添加一个指向这个新 inode 的目录项。

```
/* 目录项结构（保持不变，硬链接通过此结构实现） */
struct sfs_disk_entry {
    uint32_t ino;                // inode编号
    char name[SFS_MAX_FNAME_LEN + 1]; // 文件名
};
```

3.4 路径解析上下文

路径解析是文件系统最核心的操作之一。这个过程从根目录开始，逐级查找每个路径分量（home → user → document.txt），直到找到最终的目标。

符号链接的引入使路径解析变得复杂。当解析过程中遇到一个符号链接时，需要读取链接的目标路径，然后"跳转"到那个路径继续解析。如果目标路径是绝对路径，需要从根目录重新开始；如果是相对路径，则基于符号链接所在的目录继续。更复杂的是，符号链接的目标可能也是符号链接，形成链条甚至环路。

1. `nameidata` 结构体用于在路径解析过程中保存上下文信息。`path` 指向当前正在解析的路径字符串，`dir` 指向当前所在目录的 inode，`target` 将存储最终找到的目标 inode。

2. `symlink_depth` 是关键的安全机制：它记录当前已经跟随了多少层符号链接，每进入一个符号链接就加 1，退出时减 1。当深度超过 `MAX_SYMLINK_DEPTH` 时，返回 `ELOOP` 错误，防止符号链接循环导致的无限递归。

3. `flags` 字段控制路径解析的行为：

- `LOOKUP_FOLLOW` 是默认行为，表示跟随符号链接；
- `LOOKUP_NOFOLLOW` 表示不跟随最终路径分量的符号链接，用于 `lstat()` 和 `readlink()` 等需要操作符号链接本身而非其目标的系统调用；
- `LOOKUP_DIRECTORY` 要求最终目标必须是目录；
- `LOOKUP_CREATE` 用于创建文件的场景，允许最终路径分量不存在。

```
#define MAX_SYMLINK_DEPTH    8                // 最大符号链接嵌套深度

/* 路径解析上下文（防止符号链接循环） */
struct nameidata {
    char *path;                // 当前解析路径
    struct inode *dir;         // 当前目录inode
    struct inode *target;      // 目标inode

    int symlink_depth;         // 当前符号链接深度
    int flags;                 // 解析标志
```

```

    semaphore_t lock;                // 解析过程锁
};

/* 解析标志 */
#define LOOKUP_FOLLOW      0x01    // 跟随符号链接
#define LOOKUP_DIRECTORY  0x02    // 目标必须是目录
#define LOOKUP_CREATE      0x04    // 若不存在则创建
#define LOOKUP_NOFOLLOW    0x08    // 不跟随最终的符号链接

```

3.5 链接操作结构

`inode_operations` 结构体定义了 `inode` 级别的操作接口，是 VFS 层与具体文件系统实现之间的抽象边界。VFS 层定义了统一的接口，每个具体的文件系统提供自己的实现函数。为了支持链接机制，我们需要在这个结构体中添加四个新的函数指针。

- `link` 函数用于创建硬链接。
- `symlink` 函数用于创建符号链接。
- `readlink` 函数读取符号链接的目标路径。它是 `readlink()` 系统调用的底层实现，将符号链接存储的路径字符串复制到用户提供的缓冲区。
- `follow_link` 函数是路径解析中处理符号链接的核心：它读取符号链接的目标路径，并返回解析后的目标 `inode`。这个函数需要与 `nameidata` 配合，正确处理相对路径和绝对路径，并维护符号链接深度计数。

```

/* VFS层链接操作接口 */
struct inode_operations {
    // ... 原有操作 ...

    /* 硬链接操作 */
    int (*link)(struct inode *old_inode, struct inode *dir,
                const char *name);

    /* 符号链接操作 */
    int (*symlink)(struct inode *dir, const char *name,
                  const char *target);

    /* 读取符号链接目标 */
    int (*readlink)(struct inode *inode, char *buf, size_t buflen);

    /* 跟随符号链接 */
    struct inode *(*follow_link)(struct inode *inode,
                                struct nameidata *nd);
};

```

四、接口设计

4.1 硬链接接口

`sys_link` 是创建硬链接的系统调用入口。它接收两个路径参数：`oldpath` 是已存在文件的路径，`newpath` 是新链接的路径。调用成功后，`newpath` 将成为 `oldpath` 所指文件的另一个名字。两个路径指向同一个 `inode`，对任何一个路径的读写操作都会影响同一份数据。

硬链接有几个重要的限制，这些限制在实现中需要严格检查：首先，不能对目录创建硬链接。这是为了防止目录树形成环路。其次，不能跨文件系统创建硬链接。因为硬链接的本质是让目录项指向同一个inode，而inode编号只在单个文件系统内有意义。

```
/**
 * sys_link - 创建硬链接
 * @oldpath: 已存在文件的路径
 * @newpath: 新链接的路径
 *
 * 语义：为oldpath指定的文件创建一个新的目录项newpath，
 *       两者指向同一个inode，nlinks计数加1。
 * 限制：不能对目录创建硬链接；不能跨文件系统
 * 返回：成功返回0，失败返回错误码
 */
int sys_link(const char *oldpath, const char *newpath);
```

`vfs_link` 是 VFS 层的实现，它作为系统调用和具体文件系统之间的中间层。VFS 层负责路径解析、权限检查等通用逻辑，然后调用具体文件系统的 `link` 操作。

```
/**
 * vfs_link - VFS层硬链接实现
 * @old_inode: 源文件的inode
 * @dir: 新链接所在目录的inode
 * @new_dentry: 新目录项
 *
 * 语义：在dir目录下创建指向old_inode的新目录项
 */
int vfs_link(struct inode *old_inode, struct inode *dir,
             struct dentry *new_dentry);
```

`sfs_link` 是 SFS 文件系统的具体实现，它执行实际的磁盘操作：在目录中添加新条目，增加inode的 `nlinks` 计数，更新 `ctime` 时间戳，并将修改的inode标记为"脏"以便写回磁盘。

```
/**
 * sfs_link - SFS文件系统硬链接实现
 * @old_inode: 源文件inode
 * @dir: 目标目录inode
 * @name: 新文件名
 *
 * 语义：
 * 1. 检查old_inode不是目录
 * 2. 检查dir和old_inode在同一文件系统
 * 3. 在dir中创建新目录项，ino指向old_inode
 * 4. 增加old_inode的nlinks计数
 */
int sfs_link(struct inode *old_inode, struct inode *dir,
             const char *name);
```


4.2 符号链接接口

`sys_symlink` **创建符号链接文件**。与硬链接不同的是，`target` 参数指定的路径在创建时不需要存在——符号链接只是记录这个路径字符串，实际的路径解析发生在访问符号链接时。这种设计提供了很大的灵活性：可以创建指向尚未存在文件的符号链接，可以创建指向其他文件系统的符号链接，甚至可以创建指向网络路径的符号链接。

```
/**
 * sys_symlink - 创建符号链接
 * @target: 目标路径（可以不存在）
 * @linkpath: 符号链接文件的路径
 *
 * 语义：创建一个符号链接文件，其内容为target路径字符串
 * 返回：成功返回0，失败返回错误码
 */
int sys_symlink(const char *target, const char *linkpath);
```

`sys_readlink` **读取符号链接的内容**。注意这个系统调用读取的是符号链接文件本身存储的路径字符串，而不是跟随链接去读取目标文件的内容。返回值是实际读取的字节数；如果目标路径比提供的缓冲区长，路径会被截断。

```
/**
 * sys_readlink - 读取符号链接内容
 * @path: 符号链接文件路径
 * @buf: 用于存储目标路径的缓冲区
 * @bufsize: 缓冲区大小
 *
 * 语义：读取符号链接文件存储的目标路径
 * 返回：成功返回读取的字节数，失败返回错误码
 */
ssize_t sys_readlink(const char *path, char *buf, size_t bufsize);
```

`vfs_symlink` **是 VFS层符号链接的具体实现**。

```
/**
 * vfs_symlink - VFS层符号链接实现
 * @dir: 链接所在目录
 * @dentry: 新目录项
 * @target: 目标路径字符串
 */
int vfs_symlink(struct inode *dir, struct dentry *dentry,
                const char *target);
```

`sfs_symlink` **是 SFS 文件系统创建符号链接的具体实现**。它首先分配一个新的 inode，将其类型设为 `SFS_TYPE_LINK`。然后根据目标路径的长度选择存储方式：如果长度小于 60 字节，使用内联存储；否则分配数据块。路径存储完成后，在目标目录中创建指向新 inode 的目录项。

```
/**
 * sfs_symlink - SFS文件系统符号链接实现
 * @dir: 目录inode
 * @name: 链接文件名
 * @target: 目标路径
```



```

*
* 语义：
*   1. 分配新的inode，类型设为SFS_TYPE_LINK
*   2. 将target路径存入inode数据区
*   3. 在dir中创建目录项
*/
int sfs_symlink(struct inode *dir, const char *name,
               const char *target);

/**
* sfs_readlink - 读取符号链接目标
* @inode: 符号链接的inode
* @buf: 缓冲区
* @buflen: 缓冲区长度
*/
int sfs_readlink(struct inode *inode, char *buf, size_t buflen);

```

`sfs_follow_link` 是路径解析中处理符号链接的关键函数。它读取符号链接存储的目标路径，然后根据路径是绝对还是相对来决定下一步：如果是绝对路径（以 `/` 开头），将当前目录切换到根目录；如果是相对路径，保持在符号链接所在的目录。然后递归调用路径解析函数继续处理。这个函数需要正确维护 `nameidata` 中的 `symlink_depth`，在进入符号链接时加 1，退出时减 1，超限时返回 `ELOOP`。

```

/**
* sfs_follow_link - 解析符号链接
* @inode: 符号链接inode
* @nd: 路径解析上下文
*
* 语义：获取符号链接目标，进行路径解析
* 返回：目标inode
*/
struct inode *sfs_follow_link(struct inode *inode,
                             struct nameidata *nd);

```

4.3 通用文件操作扩展

链接机制的引入还需要对一些现有的系统调用进行扩展或修改。

`sys_unlink` 删除一个文件名。对于硬链接，这意味着从目录中移除指定的条目，并将目标 inode 的 `nlinks` 计数减 1。

```

/**
* sys_unlink - 删除文件/链接
* @path: 文件路径
*
* 语义：删除目录项，减少nlinks计数
*       当nlinks为0且无进程打开时释放inode
* 注意：对于软链接，删除链接文件本身
*/
int sys_unlink(const char *path);

```

`sys_lstat` 是 `stat` 的变体，专门用于获取符号链接本身的信息。`lstat` 不跟随符号链接，返回链接文件本身的元数据）。

```

/**
 * sys_lstat - 获取文件状态（不跟随符号链接）
 * @path: 文件路径
 * @stat: 状态结构体
 *
 * 语义：与stat类似，但对符号链接返回链接本身的信息
 */
int sys_lstat(const char *path, struct stat *stat);

```

五、同步互斥问题处理

5.1 inode 引用计数保护

硬链接操作涉及对 inode 元数据的修改，以及对目录内容的修改。这两个操作可能被多个进程并发执行，例如两个进程同时为同一个文件创建硬链接，或者一个进程创建链接的同时另一个进程在删除链接。如果没有适当的同步机制，可能出现严重的数据不一致问题。

我们采用细粒度的锁策略：每个 inode 有自己的信号量 `i_sem`，每个目录操作需要获取目录 inode 的锁。在创建硬链接时，**需要同时持有两个锁：目标目录的锁和源文件 inode 的锁。**为了避免死锁，必须规定统一的加锁顺序——我们选择**"先锁目录，后锁文件"**。所有涉及目录和文件的操作都必须遵循这个顺序。

在 `sfs_link` 函数中，我们首先获取目录锁，然后获取源文件 inode 的锁。在持有锁的情况下进行所有检查：确认源文件不是目录，确认链接数未超过上限。检查通过后，在目录中添加新条目，增加 `nlinks` 计数，更新 `ctime`。**最后按相反的顺序释放锁：先释放文件锁，后释放目录锁。**

`mark_inode_dirty(old_inode)` 调用将 inode 标记为"脏"，告诉文件系统该 inode 的内存副本已被修改，需要在适当时机写回磁盘。这个标记很重要：如果系统崩溃前没有将修改写回磁盘，`nlinks` 计数可能与实际的目录项数量不一致，导致文件系统损坏。

```

/* 硬链接创建时的同步处理 */
int sfs_link(struct inode *old_inode, struct inode *dir,
             const char *name) {
    // 获取目录锁
    down(&dir->i_sem);

    // 获取源inode锁
    down(&old_inode->i_sem);

    // 检查是否为目录
    if (S_ISDIR(old_inode->i_mode)) {
        up(&old_inode->i_sem);
        up(&dir->i_sem);
        return -EPERM;
    }

    // 检查链接数是否超限
    if (old_inode->i_nlinks >= MAX_LINKS) {
        up(&old_inode->i_sem);
        up(&dir->i_sem);
        return -EMLINK;
    }

    // 在目录中添加新条目

```

```

int ret = dir_add_entry(dir, name, old_inode->i_ino);
if (ret == 0) {
    old_inode->i_nlinks++;    // 增加链接计数
    old_inode->i_ctime = current_time();
    mark_inode_dirty(old_inode);
}

up(&old_inode->i_sem);
up(&dir->i_sem);

return ret;
}

```

5.2 目录操作的互斥

目录是一种特殊的文件，其内容是目录项的数组。多个进程可能同时对同一目录进行操作：创建新文件、删除文件、重命名文件、创建链接等。这些操作都会修改目录的内容，必须互斥执行以保证一致性。

`dir_add_entry` 函数在目录中添加一个新条目。函数开头的 `assert(holding(&dir->i_sem))` 是一个运行时检查，确保调用者已经持有目录的锁，将锁的获取和释放集中到上层函数，避免重复加锁的开销和潜在的死锁风险。

在添加条目之前，首先检查同名文件是否已存在。这个检查和随后的添加操作必须是原子的：如果在检查和添加之间有另一个进程插入了同名文件，就会出现重复条目，破坏目录的一致性。持有目录锁保证了这种原子性。检查通过后，构造新的目录项结构并写入目录文件。

```

/* 目录修改的互斥保护 */
int dir_add_entry(struct inode *dir, const char *name, uint32_t ino) {
    // 调用者需持有dir->i_sem
    assert(holding(&dir->i_sem));

    // 检查名称是否已存在
    if (dir_lookup(dir, name, NULL) == 0) {
        return -EEXIST;
    }

    // 分配新的目录项空间
    struct sfs_disk_entry entry;
    entry.ino = ino;
    strncpy(entry.name, name, SFS_MAX_FNAME_LEN);

    // 写入目录
    return sfs_write_entry(dir, &entry);
}

```

5.3 符号链接解析的循环检测

符号链接可以形成任意复杂的链条，一个符号链接指向另一个符号链接，后者又指向第三个，以此类推。更危险的是，符号链接可能形成环路，如果不加限制地跟随符号链接，这种环路会导致路径解析陷入无限循环，最终耗尽栈空间或其他系统资源。

`resolve_path` 函数展示了如何在路径解析中安全地处理符号链接。核心机制是 `symlink_depth` 计数器：每次进入一个符号链接就加 1，处理完退出时减 1。当深度超过 `MAX_SYMLINK_DEPTH` 时，立即停止解析并返回 `ELOOP` 错误。这个限制不仅防止了环路，也限制了过长的链接链条，因为过长的链条会严重影响性能。

整个解析过程由 `nd->lock` 信号量保护。虽然路径解析本身通常可以并发进行，但单个解析过程中的状态必须保持一致，不能被其他线程打断。

```
/* 路径解析中的符号链接处理 */
struct inode *resolve_path(const char *path, struct nameidata *nd) {
    struct inode *inode = NULL;
    char *component;

    // 初始化解析上下文
    nd->symlink_depth = 0;
    down(&nd->lock);

    while ((component = next_component(path)) != NULL) {
        inode = dir_lookup(nd->dir, component);

        if (inode == NULL) {
            up(&nd->lock);
            return NULL;
        }

        // 检查是否为符号链接
        if (S_ISLNK(inode->i_mode) && (nd->flags & LOOKUP_FOLLOW)) {
            // 检查嵌套深度
            if (nd->symlink_depth >= MAX_SYMLINK_DEPTH) {
                iput(inode);
                up(&nd->lock);
                return ERR_PTR(-ELOOP); // 检测到循环
            }

            nd->symlink_depth++;

            // 读取符号链接目标
            char target[MAX_SYMLINK_LEN];
            sfs_readlink(inode, target, sizeof(target));

            // 释放当前inode，解析目标路径
            iput(inode);

            // 递归解析（处理绝对/相对路径）
            if (target[0] == '/') {
                nd->dir = get_root_inode();
            }
            inode = resolve_path(target, nd);

            nd->symlink_depth--;
        }

        if (S_ISDIR(inode->i_mode)) {
            iput(nd->dir);
            nd->dir = inode;
        }
    }
}
```

```

    }
}

up(&nd->lock);
return inode;
}

```

5.4 删除操作的同步处理

`sys_unlink` 的实现展示了删除文件或链接时的完整同步流程。删除操作涉及两个对象：父目录和目标文件的 `inode`。两者都可能被其他进程并发访问，因此都需要加锁保护。

函数首先调用 `path_to_parent` 解析路径，获取目标文件所在的目录 `inode` 和文件名。然后按照“先目录后文件”的顺序获取锁，在目录中查找目标文件的 `inode`。

在持有两把锁的情况下，进行一系列检查。如果目标是目录，返回 `EISDIR` 错误——目录应该使用 `rmdir` 删除，而不是 `unlink`。检查通过后，从目录中移除条目，减少 `inode` 的 `nlinks` 计数。如果 `nlinks` 降为 0，设置 `I_FREEING` 标志，但不立即删除 `inode` 和数据块。

这种“延迟删除”机制是 UNIX 文件语义的重要组成部分。即使文件的所有链接都被删除，只要还有进程打开了这个文件，进程就可以继续读写文件。只有当最后一个引用被释放时，文件的数据块才会被真正回收。`iput(inode)` 调用会检查这些条件，在适当时机触发实际删除。

```

/* unlink操作的同步处理 */
int sys_unlink(const char *path) {
    struct inode *dir, *inode;
    char name[SFS_MAX_FNAME_LEN + 1];

    // 解析路径，获取父目录和文件名
    dir = path_to_parent(path, name);
    if (dir == NULL) {
        return -ENOENT;
    }

    down(&dir->i_sem);

    // 查找目标inode
    inode = dir_lookup(dir, name);
    if (inode == NULL) {
        up(&dir->i_sem);
        iput(dir);
        return -ENOENT;
    }

    down(&inode->i_sem);

    // 不允许删除目录（应使用rmdir）
    if (S_ISDIR(inode->i_mode)) {
        up(&inode->i_sem);
        up(&dir->i_sem);
        iput(inode);
        iput(dir);
        return -EISDIR;
    }
}

```

```

// 从目录中删除条目
int ret = dir_remove_entry(dir, name);
if (ret == 0) {
    inode->i_nlinks--;
    inode->i_ctime = current_time();

    // 如果链接计数为0，标记为待删除
    if (inode->i_nlinks == 0) {
        // 延迟删除：等待所有引用释放后真正删除
        inode->i_flags |= I_FREEING;
    }
    mark_inode_dirty(inode);
}

up(&inode->i_sem);
up(&dir->i_sem);

iput(inode); // 可能触发实际删除
iput(dir);

return ret;
}

```

六、实现要点总结

6.1 硬链接实现要点

- 链接计数管理：**`nlinks` 字段是硬链接机制的核心。必须在持有锁的情况下原子地更新这个计数，确保它始终反映实际指向该 inode 的目录项数量。计数不准确会导致文件过早删除（数据丢失）或永不删除（空间泄漏）。
- 跨文件系统检查：**硬链接不能跨越文件系统边界。实现时需要比较源文件和目标目录的超级块指针或设备号，确保它们属于同一个文件系统。尝试跨文件系统创建硬链接应返回 `EXDEV` 错误。
- 目录限制：**禁止对目录创建硬链接是维护文件系统树形结构的关键。目录的 `nlinks` 有特殊含义：它等于 2 加上子目录的数量。允许任意硬链接会破坏这个不变式，导致各种问题。
- 延迟删除：**当 `nlinks` 为 0 时，不能立即释放 inode 和数据块，因为可能还有进程持有该文件的打开描述符。需要设置标志，在最后一个引用释放时才执行实际删除。

6.2 符号链接实现要点

- 路径存储：**采用内联存储优化短路径的访问性能。60 字节的阈值是根据 inode 结构的剩余空间选择的，可以容纳大多数常见的相对路径和短绝对路径。
- 循环检测：**符号链接深度限制是防止无限循环的关键安全机制。限制值的选择需要平衡安全性和可用性：太小会阻止合法的深层链接，太大会在遇到循环时消耗过多资源。8 到 40 之间是常见的选择。
- 悬空链接处理：**符号链接的目标可能不存在。这不是错误——`symlink()` 调用应该成功。只有在实际访问悬空链接时才返回 `ENOENT`。`lstat()` 和 `readlink()` 对悬空链接仍然有效。
- 相对路径处理：**符号链接中的相对路径应基于链接所在的目录解析，而非进程的当前工作目录。这是 POSIX 标准规定的语义。实现时，在开始解析目标路径前，需要将"当前目录"设置为符号链接所在的目录。

总结

本实验报告详细设计了 uCore 操作系统中 UNIX PIPE 机制和软硬链接机制的实现方案，对每个数据结构和接口都给出了详细的描述和设计理由。

对于 PIPE 机制，我们设计了以下核心组件：

- `pipe_inode` 结构体，包含 4KB 环形缓冲区和完整的同步原语
- 扩展的 `file` 结构体，支持管道文件类型
- `pipe_operations` 函数指针表，实现与 VFS 的统一接口
- 完整的系统调用接口（`sys_pipe`）和内核内部接口
- 基于信号量的同步机制，正确处理生产者-消费者问题
- 引用计数管理，处理管道关闭时的唤醒和资源释放

对于软硬链接机制，我们设计了以下核心组件：

- 扩展的 `sfs_disk_inode` 结构体，包含链接计数和文件类型
- 符号链接的内联存储优化
- `nameidata` 路径解析上下文，支持符号链接深度限制
- 完整的系统调用接口（`sys_link`、`sys_symlink`、`sys_readlink`、`sys_lstat`）
- 统一的加锁顺序（先目录后文件），避免死锁
- 延迟删除机制，支持 UNIX "删除已打开文件"的语义

两个机制的设计都充分考虑了同步互斥问题，通过信号量、互斥锁和引用计数等机制，确保在多进程并发环境下的正确性和数据一致性。设计参考了 Linux 内核的实现，同时针对 uCore 的特点进行了适当简化，在保持 UNIX 语义正确性的同时降低了实现复杂度。